

Database Backup Utility

Full Project Documentation & Learning Reference

A command-line tool for backing up and restoring SQLite, MySQL, and MongoDB databases — built from scratch as a hands-on learning project covering database internals, backend architecture, subprocess management, compression, logging, testing, and scheduling.

Project page: <https://roadmap.sh/projects/database-backup-utility>

Repository: <https://github.com/HADIL19/database-backup-utility>

This document is a complete reference to what was built, why each decision was made, every tool and library used and what it's for, how testing works, and the real bugs encountered and fixed along the way. Written to be revisited before interviews or shared with others.

Table of Contents

1. Project Overview
2. Architecture & Design Philosophy
3. Tools and Libraries Used — What and Why
4. Database Connectors (SQLite, MySQL, MongoDB)
5. Compression
6. Local Storage
7. Logging
8. Scheduling
9. Testing (pytest)
10. Real Bugs Encountered and How They Were Fixed
11. Git Workflow Used
12. Key Concepts Explained (for review)
13. What's Left / Roadmap
14. How to Talk About This Project in Interviews

1. Project Overview

This project is a CLI (command-line interface) tool that backs up and restores databases. It supports three different database engines — SQLite, MySQL, and MongoDB — through a single, consistent command-line interface.

Why it matters as a learning project: most simple backup scripts are a single function calling a dump tool and hoping for the best. This project instead treats backup/restore as a reliability-critical operation, the way it's actually handled in production systems — with consistency guarantees, proper error handling, structured logging, and an architecture that scales to new database engines without rewriting existing code.

Core capabilities

- Back up and restore SQLite, MySQL, and MongoDB databases
- Automatic compression of backup files (gzip for single files, zip for MongoDB's folder-based dumps)
- Structured logging of every operation (start time, end time, duration, status, errors)
- Local storage abstraction, designed to extend to cloud storage later
- Scheduled/automatic recurring backups
- Secure credential handling (passwords never exposed in process listings)
- Automated test suite (pytest) proving backup/restore actually works, not just that it runs

2. Architecture & Design Philosophy

The single most important design decision in this project is the use of a **shared interface** (an abstract base class) that every database connector must implement. This is a real, named software design pattern called the **Strategy Pattern** — swap the implementation, keep the interface the same.

Folder structure

```
db_backup_utility/  
|-- cli.py                # Entry point -- argument parsing and orchestration o  
nly  
|-- connectors/  
|   |-- base.py           # Abstract DBConnector interface  
|   |-- sqlite.py         # SQLite implementation  
|   |-- mysql.py          # MySQL implementation (mysqldump/mysql)  
|   |-- mongo.py          # MongoDB implementation (mongodump/mongorestore)
```

```

|-- backup/
|  |-- compression.py      # gzip (files) / zip (folders) compress & decompress
|-- storage/
|  |-- local.py            # Local disk storage abstraction
|-- logging_utils/
|  |-- logger.py           # Centralized structured logging
|-- tests/
|  |-- test_sqlite_connector.py
|  |-- test_mongo_connector.py
|  |-- test_compression.py
|-- make_test_db.py        # Helper: generates sample SQLite test data
`-- requirements.txt

```

The core rule: cli.py never touches database logic

cli.py only does three things: parse command-line arguments (via Click), pick the correct connector using a factory function, and call the same three methods on it — `test_connection()`, `backup()`, `restore()` — regardless of which database is being used. This means adding a new database engine (e.g. PostgreSQL later) requires writing exactly one new class and never touching cli.py's logic.

The abstract base class (connectors/base.py)

```

from abc import ABC, abstractmethod

class DBConnector(ABC):
    @abstractmethod
    def test_connection(self) -> bool:
        pass

    @abstractmethod
    def backup(self, output_path: str) -> str:
        pass

    @abstractmethod
    def restore(self, backup_path: str) -> None:
        pass

```

ABC (Abstract Base Class) means this class can never be instantiated directly — it only exists to be inherited from. `@abstractmethod` forces every subclass (`SQLiteConnector`, `MySQLConnector`, `MongoConnector`) to implement all three methods, or Python refuses to create an instance of that class at all. This isn't just theory — it caught a real bug during development (see section 10).

The factory function (in cli.py)

```
def get_connector(db_type, path, host, port, user, password, database):  
    if db_type == 'sqlite':  
        return SQLiteConnector(path)  
    elif db_type == 'mysql':  
        return MySQLConnector(host, port, user, password, database)  
    elif db_type == 'mongo':  
        return MongoConnector(host, port, database)  
    else:  
        return None
```

This single function is the only place that knows about all three connector types. Everywhere else in the code just calls the returned object's methods without caring what database it actually is.

3. Tools and Libraries Used — What and Why

Tool / Library	What it is	Why it's used here
Python 3.10+	The programming language	Chosen for its strong database libraries and easy subprocess handling
Click	CLI argument-parsing library	Turns Python functions into command-line commands with automatic --help text, validation, and type checking
sqlite3	Python's built-in SQLite library	Used for SQLite connections and its safe backup() API (not a raw file copy)
mysqldump / mysql	MySQL's official CLI tools	Used via subprocess to create and restore MySQL backups — the same tools real DBAs use
mongodump / mongorestore	MongoDB's official CLI tools (MongoDB Database Tools)	Used via subprocess to export/import MongoDB data as BSON files
pymongo	Python's MongoDB driver	Used for connection testing (ping) and for creating/verifying test data directly in Python
gzip / zipfile (via shutil)	Python's built-in compression modules	gzip compresses single files (SQLite/MySQL backups); zip compresses MongoDB's folder-based backups
schedule	Lightweight Python job-scheduling library	Powers the schedule-backup command, running backups on a recurring interval
logging (built-in)	Python's structured logging module	Records every backup/restore operation with timestamps, severity levels, and messages, to both console and a log file
pytest	Python's most popular testing framework	Automatically runs test functions and reports pass/fail, replacing manual re-testing
subprocess	Python's built-in module for running external programs	Used to call mysqldump, mysql, mongodump, and mongorestore as separate processes and capture their output
Git / GitHub	Version control and code hosting	Tracks all changes; used with a branch → pull request → merge workflow

Docker (considered, not used)	Containerization platform	Initially considered for running MySQL/MongoDB in containers; not available on this machine, so native installs were used instead
-------------------------------	---------------------------	---

4. Database Connectors

4.1 SQLite Connector

SQLite stores an entire database as a single file — there's no separate server process. The naive way to 'back it up' would be to copy the file with `shutil.copy()`, but this is unsafe: if the file is being written to at the exact moment of copying, the backup can end up corrupted or inconsistent.

Solution used: Python's built-in `sqlite3` module provides a native `backup()` API that copies data page-by-page and guarantees a consistent snapshot, even if the source database is actively being written to.

```
def backup(self, output_path: str) -> str:
    source_conn = sqlite3.connect(self.path)
    dest_conn = sqlite3.connect(output_path)
    with dest_conn:
        source_conn.backup(dest_conn)
    source_conn.close()
    dest_conn.close()
    return output_path
```

`restore()` works identically but in reverse — the backup file becomes the source, and the live database becomes the destination.

4.2 MySQL Connector

MySQL is a running server, not a single file, so backup means exporting the data as SQL statements that can recreate it. The standard, production-grade tool for this is `mysqldump` — this project wraps it via Python's `subprocess` module rather than reimplementing dump logic from scratch.

Key flag used: `--single-transaction`, which ensures the dump reflects one consistent snapshot of the database, even under concurrent writes, without locking the whole database for other users.

Security concern addressed: passing a password directly on the command line (e.g. `--password=xxx`) exposes it to anyone who can view running processes (`ps aux` on Linux, Task Manager on Windows). Instead, the password is passed via the `MYSQL_PWD` environment variable, which `mysqldump` reads securely without it ever appearing in a process listing.

```
def _env(self):
    env = os.environ.copy()
    env["MYSQL_PWD"] = self.password
    return env

def backup(self, output_path: str) -> str:
```



```

with open(output_path, "w") as f_out:
    result = subprocess.run(
        ["mysqldump", "-h", self.host, "-P", str(self.port),
         "-u", self.user, "--single-transaction", self.database],
        env=self._env(), stdout=f_out, stderr=subprocess.PIPE, text=True
    )
if result.returncode != 0:
    raise RuntimeError(f"mysqldump failed: {result.stderr}")
return output_path

```

Notice `stdout=f_out` streams `mysqldump`'s output directly into a file as it's produced, instead of loading the entire dump into memory first — this matters for handling large databases efficiently.

4.3 MongoDB Connector

MongoDB stores data as JSON-like 'documents' inside 'collections' (loosely: tables), inside a database — no upfront schema (`CREATE TABLE`) is required. Its backup tool, `mongodump`, exports data as BSON (Binary JSON) files.

Important structural difference from SQLite/MySQL: `mongodump` produces a whole DIRECTORY of files (one `.bson` and one `.metadata.json` per collection), not a single file. This required the connector's `backup()` to return a folder path instead of a file path, and it required the compression logic to be updated to handle folders (see section 5).

```

def backup(self, output_path: str) -> str:
    os.makedirs(output_path, exist_ok=True)
    result = subprocess.run(
        ["mongodump", f"--host={self.host}", f"--port={self.port}",
         f"--db={self.database}", f"--out={output_path}"],
        capture_output=True, text=True
    )
    if result.returncode != 0:
        raise RuntimeError(f"mongodump failed: {result.stderr}")
    return output_path

def restore(self, backup_path: str) -> None:
    db_dump_path = os.path.join(backup_path, self.database)
    result = subprocess.run(
        ["mongorestore", f"--host={self.host}", f"--port={self.port}",
         f"--db={self.database}", "--drop", db_dump_path],
        capture_output=True, text=True
    )
    if result.returncode != 0:
        raise RuntimeError(f"mongorestore failed: {result.stderr}")

```

The `--drop` flag ensures existing collections are dropped before restoring, preventing duplicate documents from being created if the collection already has data in it — this guarantees an exact, clean restore matching the backup precisely.

`test_connection()` for MongoDB has a fallback: it first tries the `mongosh` shell if available, and if not found, falls back to a lightweight ping using the `pymongo` driver directly. This is a defensive pattern — never assume a specific external tool is available, degrade gracefully instead.

5. Compression

Backups need to be compressed to save storage space. The design principle followed here: compress with chunked, streaming I/O so large files never need to be fully loaded into memory at once.

Format-aware compression: the key design decision

Because SQLite and MySQL backups are single files, but MongoDB backups are folders, a single `compress_file()` function was built that auto-detects which kind of input it received and branches accordingly:

- Single file → compressed with Python's built-in `gzip` module, producing a `.gz` file
- Folder → compressed with `shutil.make_archive()` into a standard `.zip` file

```
def compress_file(source_path: str, remove_original: bool = True) -> str:
    if os.path.isdir(source_path):
        archive_base = source_path.rstrip(os.sep)
        archive_path = shutil.make_archive(archive_base, 'zip', source_path)
        if remove_original:
            shutil.rmtree(source_path)
        return archive_path
    else:
        compressed_path = source_path + ".gz"
        with open(source_path, 'rb') as f_in:
            with gzip.open(compressed_path, 'wb') as f_out:
                shutil.copyfileobj(f_in, f_out)
        if remove_original:
            os.remove(source_path)
        return compressed_path
```

Because both branches are hidden behind the same function name, none of the calling code (in `cli.py`) needs to know or care whether it's dealing with SQLite, MySQL, or MongoDB output — it just calls `compress_file(path)` and gets back a compressed result, regardless of format. This is the same 'swap the implementation, keep the interface' idea used for the database connectors, applied here to compression.

`shutil.copyfileobj()` copies data between files in chunks, rather than reading whole files into memory — this is what makes the tool able to handle large databases without excessive memory use.

6. Local Storage

The 'where do backup files actually get saved' logic was pulled out of cli.py into its own module, storage/local.py, with a LocalStorage class exposing one method: save().

```
class LocalStorage:
    def __init__(self, base_dir: str):
        self.base_dir = base_dir
        os.makedirs(self.base_dir, exist_ok=True)

    def save(self, source_path: str) -> str:
        filename = os.path.basename(source_path)
        dest_path = os.path.join(self.base_dir, filename)
        if os.path.abspath(source_path) != os.path.abspath(dest_path):
            shutil.move(source_path, dest_path)
        return dest_path
```

This exists specifically so that a future S3Storage or GCSSStorage class can implement the exact same save() method, but upload to the cloud instead of moving a local file — and cli.py would not need to change at all to support it.

7. Logging

Every backup and restore operation logs structured information: start time, completion status, duration, and any errors — to both the console (for live feedback) and a persistent file, logs/backup_activity.log (for later auditing).

```
def get_logger():
    logger = logging.getLogger('db_backup_utility')
    logger.setLevel(logging.INFO)
    if not logger.handlers:
        file_handler = logging.FileHandler('logs/backup_activity.log')
        console_handler = logging.StreamHandler()
        formatter = logging.Formatter(
            '%(asctime)s | %(levelname)s | %(message)s',
            datefmt='%Y-%m-%d %H:%M:%S'
        )
        file_handler.setFormatter(formatter)
        console_handler.setFormatter(formatter)
        logger.addHandler(file_handler)
        logger.addHandler(console_handler)
    return logger
```

Example log output:

```
2026-08-15 09:02:41 | INFO | Backup started | db_type=sqlite
2026-08-15 09:02:41 | INFO | Backup completed | file=./backups/test.db.bak.gz | d
```

```
uration=0.04s
```

Log levels (INFO, WARNING, ERROR) matter because they let you filter later — e.g. 'show only failures from the last week' — and many real monitoring/alerting tools trigger specifically on ERROR-level entries.

Every backup/restore operation is also wrapped in a try/except Exception block. This is different from the explicit checks (like 'does this file exist') — the try/except catches unknown, unanticipated failures (disk full, permission errors, network issues) and logs them cleanly instead of crashing with a raw Python traceback.

8. Scheduling

A `schedule-backup` CLI command runs backups automatically on a recurring interval, using the `schedule` library, without needing any external tooling.

Important refactor that made this possible: the core backup logic was extracted out of the `backup` Click command into a standalone function, `run_backup()`, so that both the manual backup command and the scheduler call the exact same logic. This avoids duplicating logic in two places that could silently drift out of sync over time.

```
def schedule_backup(...):
    def job():
        success = run_backup(db_type, path, host, port, user,
                             password, database, output, compress)
        ...

    schedule.every(interval_minutes).minutes.do(job)
    job() # run once immediately

    while True:
        schedule.run_pending()
        time.sleep(1)
```

`schedule.run_pending()` must be called repeatedly in a loop — it checks whether it's time to run any scheduled job. `time.sleep(1)` pauses briefly between checks so the loop doesn't spin the CPU at 100%.

Honest limitation, documented in the README

This scheduler only runs while its terminal/process stays open — it is not a persistent background service. For real production use, running the backup command via `cron` (Linux/Mac) or `Task Scheduler` (Windows) is more robust, since those survive reboots without needing an open terminal. Both approaches are documented, and the tradeoff between them is understood and explained rather than hidden.

9. Testing (pytest)

Manual testing (running commands and eyeballing output) is slow and error-prone — it's exactly how a real bug slipped through earlier in this project (see section 10). `pytest` automates this: it runs Python functions starting with `test_`, and reports pass/fail based on `assert` statements inside them.

The Arrange-Act-Assert pattern

Every test in this project follows the same three-part structure, a universal pattern in software testing:

- **Arrange:** set up a known starting state (e.g. create a database with known data)
- **Act:** run the actual code being tested (e.g. `connector.backup(...)`)
- **Assert:** check the result matches what's expected

pytest fixtures

`tmp_path` is a built-in pytest fixture that gives each test function a fresh, automatically-cleaned-up temporary folder — so tests never interfere with each other or leave junk files behind. It's injected automatically just by naming it as a function parameter.

Test suite summary

Test file	What it verifies
<code>test_sqlite_connector.py</code>	Backup/restore round-trip; full disaster-recovery simulation (drop table, restore, verify data); <code>test_connection()</code> correctly fails for a missing file
<code>test_mysql</code> (not yet automated)	Currently verified manually only — would need a running MySQL instance during test runs, typically handled with a dedicated test DB or CI container
<code>test_mongo_connector.py</code>	Backup/restore round-trip with real MongoDB documents; <code>test_connection()</code> succeeds against a running server
<code>test_compression.py</code>	Compress → decompress returns the exact original bytes; <code>compress_file()</code> removes the original source file by default

Final confirmed result, running the full suite:

```
7 passed in 1.08s
```

10. Real Bugs Encountered and How They Were Fixed

These are genuine debugging stories from building this project — worth remembering and able to explain in detail, since they demonstrate real problem-solving, not just following a tutorial.

Bug 1: `decompress_file()` silently did nothing

Symptom: restore reported 'success', but the database remained empty after restoring.

Root cause: the `decompress_file()` function correctly calculated what the output path SHOULD be and returned that string, but never actually opened the compressed file or wrote any decompressed bytes to disk. The function looked complete (correct signature, correct return type) but its actual body was missing the real logic.

How it was found: methodical debugging — checking `os.path.exists()` on the supposedly-decompressed file revealed it didn't actually exist, despite no error being raised. This proved the bug was in decompression, not in the restore logic that consumed it.

Lesson: a function can have the right signature and return type and still do nothing. Always verify actual output data, not just that a function 'ran without error'. This bug is exactly what the later `pytest test_compress_and_decompress_roundtrip` test was written to catch automatically.

Bug 2: `MySQLConnector's test_connection()` method went missing

Symptom: VS Code's editor showed a red underline under a call to `MySQLConnector(...)`, with an error stating the abstract class couldn't be instantiated.

Root cause: the `test_connection()` method had been accidentally deleted from `connectors/mysql.py` at some point during editing, leaving only `backup()` and `restore()` implemented.

Why this was caught immediately: because `MySQLConnector` inherits from the abstract base class `DBConnector`, and `DBConnector` marks `test_connection()` as `@abstractmethod`, Python refused to let the class be instantiated at all until the method existed again. Instead of a confusing failure deep inside a real backup run, the bug was caught at the earliest, clearest possible point — object creation.

Lesson: this is a genuine, concrete demonstration of why abstract base classes are useful in real engineering — they don't just document intent, they actively prevent incomplete implementations from being used.

Bug 3: MySQL backup failed after a working session — but it wasn't a code bug

Symptom: after previously confirming MySQL backups worked, a later session showed 'could not connect to database' errors.

Debugging process: rather than assuming the recently-changed code was at fault, the actual MySQL server connection was tested manually and independently, outside of any Python code: `mysql -h localhost -P 3306 -u root -e "SELECT 1;"` — this returned connection error 10061 ('connection refused'), which specifically means nothing is listening on that port at all.

Root cause: the MySQL Windows service (MySQL80) had simply stopped running — nothing to do with the Python code.

Fix: restarted the service using `net start MySQL80`, run from an Administrator PowerShell (starting/stopping Windows services requires elevated permissions — running as a non-admin gave 'Access denied').

Lesson: when debugging, isolate whether the problem is in your code or in the surrounding infrastructure, before assuming your recent change caused it. Testing the raw MySQL connection independently of the Python code proved the code was innocent, saving time that would've been wasted re-reading correct code.

11. Git Workflow Used

Later in the project, direct commits to the main branch were replaced with a proper branch → pull request → merge workflow, matching how most real engineering teams operate — even for solo work, since it's a genuinely valuable habit and a common interview topic.

The workflow, step by step

```
# 1. Create a new branch for a specific change
git checkout -b feature/local-storage

# 2. Make changes, then commit them on that branch
git add .
git commit -m "Formalize local storage into storage/local.py"

# 3. Push the branch to GitHub
git push -u origin feature/local-storage

# 4. On GitHub: open a Pull Request (feature/local-storage -> main)
#    Review the diff, then click "Merge pull request"

# 5. Sync local main with the merged changes
git checkout main
git pull
```

Why this matters: it isolates work-in-progress changes from the stable main branch, creates a reviewable record of what changed and why (via the PR description), and mirrors real team workflows where code is rarely pushed directly to main without review.

git add stages changes; git commit saves a permanent snapshot locally; git push uploads commits to GitHub. None of this happens automatically — every step is a deliberate, explicit action.

12. Key Concepts Explained (for review)

Abstract Base Class (ABC)

A class that cannot be instantiated directly and defines methods that subclasses are REQUIRED to implement, enforced by Python at object-creation time. Used here (DBConnector) so every database connector guarantees the same interface.

Strategy Pattern

A design pattern where multiple interchangeable implementations share the same interface, so calling code can swap between them without changes. Applied here to database connectors

(SQLite/MySQL/MongoDB) and to storage backends (local/future cloud).

Factory function

A function whose only job is to decide which class to instantiate based on some input (here, `db_type`), keeping that decision logic in one central place instead of scattered if/else blocks throughout the codebase.

Consistency (in backups)

A guarantee that a backup reflects the database at one single point in time, even if the database is being written to during the backup — achieved via SQLite's backup API and MySQL's `--single-transaction` flag.

Streaming I/O

Processing data in small chunks rather than loading an entire file into memory at once — essential for handling large databases without exhausting available RAM. Used throughout compression and MySQL dump/restore.

Idempotency-adjacent design (--drop flag)

Ensuring a restore produces the exact same end state every time, regardless of what existed before it, by explicitly dropping old data first rather than appending to it.

Environment variables for secrets

Passing sensitive values (like database passwords) to subprocesses via the OS environment rather than command-line arguments, since command-line arguments are visible to any process-inspection tool, while environment variables passed this way are not.

Structured logging

Recording operational events with consistent fields (timestamp, severity level, message) to enable later filtering, searching, and automated alerting — as opposed to unstructured `print()` statements.

Arrange-Act-Assert

The standard three-part structure of an automated test: set up known state, run the code under test, verify the result matches expectations.

pytest fixtures

Reusable setup/teardown logic that pytest automatically provides to test functions that request it as a parameter (e.g. `tmp_path` for a clean temporary directory per test).

13. What's Left / Roadmap

The following were deliberately scoped out of this version, and documented honestly rather than hidden, since the architecture already supports adding them:

- PostgreSQL connector — same DBConnector interface as the other three, using `pg_dump/pg_restore`
- Cloud storage backends (S3 / Google Cloud Storage) — would implement the same `save()` interface as `LocalStorage`
- Slack notifications on backup completion/failure via webhook
- Selective (table/collection-level) restore, instead of always restoring the entire database
- Authentication support for MongoDB (current version assumes a local, unauthenticated instance)
- Automated MySQL tests in the pytest suite (currently verified manually only)

This honesty is intentional — the project's README documents exactly this: what's built and verified versus what's planned, so nothing is overclaimed.

14. How to Talk About This Project in Interviews

A few genuinely strong, specific talking points this project provides, worth having ready:

On architecture

"I used the Strategy pattern via an abstract base class so that SQLite, MySQL, and MongoDB could all be backed up through the same interface. Adding a new database engine only requires writing one new class — no existing code changes."

On a real bug caught by good design

"During development, a required method was accidentally deleted from one of my connector classes. Because I'd used an abstract base class with `@abstractmethod`, Python refused to let the class be instantiated at all — catching the bug immediately at the clearest possible point, instead of causing a confusing failure deep inside an actual backup run."

On testing discipline

"I don't trust a backup tool that's never had its restore path tested. Every connector was validated with an actual disaster-recovery drill — create data, back it up, deliberately destroy the live database, restore, and confirm byte-for-byte recovery — and I automated that exact scenario into my pytest suite."

On methodical debugging

"When a previously-working feature broke, instead of assuming my last code change caused it, I tested the underlying infrastructure independently first — which revealed the actual database server had stopped running, unrelated to any code change. That isolation step saved time I would've otherwise spent second-guessing correct code."

On scope and honesty

"I intentionally scoped out PostgreSQL, cloud storage, and Slack notifications for this version, and documented them as a roadmap rather than claiming they were built. The architecture supports adding them without rework, but I'd rather be precise about what's actually tested and working."